

# Web Development: HTML, CSS, and HTTP

A practical study reference with foundational concepts, examples, and common review questions.

## 1. Web and HTTP

A browser resolves a URL, sends an HTTP request, and renders the response. DNS resolves names; HTTPS uses TLS to protect HTTP. Requests contain methods, headers, and optional bodies; responses contain status codes, headers, and optional content.

GET retrieves data and should not change server state; POST processes or creates; PUT replaces; PATCH partially updates; DELETE removes. Common statuses include 200, 201, 400, 401, 403, 404, and 500. APIs define contracts between clients and services.

## 2. Semantic HTML and Accessibility

HTML describes document structure. Use semantic elements, logical headings, associated form labels, and buttons for actions. Provide meaningful alternative text for useful images; decorative images can use empty alt text. Keyboard users need visible focus and logical navigation.

Use native HTML accessibility before ARIA. Forms should describe errors, identify required fields, and announce important status changes. Do not use color alone to convey meaning.

## 3. CSS and Performance

CSS styling follows cascade, specificity, inheritance, and source order. The box model contains content, padding, border, and margin; border-box includes padding and border in declared dimensions. Flexbox handles one-dimensional layouts; Grid handles two-dimensional layouts.

Responsive design adapts to viewport and user preferences. Test narrow and wide screens, contrast, text scaling, and reduced motion. Browsers parse HTML and CSS, compute layout, paint, and composite; measure before optimizing assets or scripts.

## 4. Security and Common Questions

Same-origin policy restricts cross-origin access; CORS lets servers selectively authorize browser access and is not authentication. Prevent XSS with context-aware output encoding, protect state-changing requests against CSRF, use secure cookies, and authorize on the server.

Authentication verifies identity; authorization checks allowed actions. Responsive design adapts usable content to devices and settings. REST is an architectural style commonly applied to resource-oriented web APIs; implementations vary in how completely they follow its constraints.